

# Can Retrieval-Augmented Few-Shot Prompting Match LoRA Fine-Tuning for Issue Report Classification? An Empirical Study

Anonymous Author(s)  
Anonymous Affiliation(s)

**Abstract**—Classifying issue reports as bugs, feature requests, or questions helps developers triage them. Fine-tuned large language models (LLMs) give the best reported results, but fine-tuning needs a training run, reaches its best scores only with labeled issues pooled across projects, and must be repeated for new labels or a newer model. Retrieval-augmented few-shot prompting (RAG) instead shows the LLM the most similar labeled issues as examples and needs no training. To our knowledge, it has not been systematically evaluated for this task or compared with Low-Rank Adaptation (LoRA) fine-tuning. We conduct an empirical study with four Qwen2.5-Instruct models and 6,600 issue reports from eleven projects. RAG, like prior methods, most often labels questions as bugs, so a variant (filtered RAG) removes bug-labeled examples when the retrieved issues suggest a question. Using only the target project’s 300 labeled issues, filtered RAG comes within 1.0 point of fine-tuning on all 3,300 labeled issues of the eleven projects in macro  $F_1$ , a difference that is not statistically significant, and needs no training and 33% less peak GPU memory on average. Without the filter, RAG trails fine-tuning by 2.8 points. Given the same labeled issues, both RAG variants outperform fine-tuning at every model size. RAG’s inference is slower, and the two approaches make different errors: fine-tuning finds more bugs but over-predicts the bug label. With this filter, retrieval-augmented few-shot prompting preserves the macro  $F_1$  of LoRA fine-tuning with  $11\times$  less labeled data per project, no training, and a third less GPU memory.

**Index Terms**—Issue classification, Retrieval-augmented generation, Fine-tuning, Large language models, Mining software repositories

## I. INTRODUCTION

Users and contributors of software projects report bugs, request features, and ask questions through issue trackers such as GitHub Issues [1]. Labeling each issue report with its type speeds up its resolution [2] and supports issue prioritization [3], issue assignment [4], and developer onboarding [5], [6]. Yet labels are rarely used [7] and often wrong [8], [9], and manually assigning them is laborious [10]. To address these problems, many studies propose methods for automated issue report classification (IRC) and typically evaluate them on the labels *bug*, *feature*, and *question* [7], [11]–[14].

Most IRC methods learn from labeled issues. Early work trained classic machine-learning classifiers [7], [8], [15], later work fine-tuned pre-trained transformer models [11], [16]–[18], and recent work applies large language models (LLMs) [6], [13], [14]. Among LLM-based methods, fine-tuning on labeled issues gives the best reported results [6], [14], [19], with Low-Rank Adaptation (LoRA [20]) for open models. Fine-tuning

has costs, however. Each model needs a training run, and even LoRA training needs GPU memory beyond what inference needs. The best scores come from pooling the labeled issues of many projects into one training set [14]. The training must also be repeated to learn from newly labeled issues or to adopt a newer model [7], [10].

LLMs can also use labeled issues without training. Through in-context learning, a model infers a task from a few labeled examples in its prompt [21], and choosing the labeled examples most similar to the query issue can outperform choosing them at random [22]. With such retrieval-augmented few-shot prompting (RAG), a newly labeled issue only needs to be indexed, and the LLM can be replaced without retraining. For future work, Heo and Lee [14] suggest few-shot prompting, and Aracena et al. [6] suggest retrieval augmentation to reduce the need for extensive fine-tuning and to help with ambiguous labels such as question. To our knowledge, however, retrieval-augmented few-shot prompting has not been systematically evaluated for IRC or compared with LoRA fine-tuning.

We conduct an empirical study that does both. We use four Qwen2.5-Instruct models of 3B to 32B parameters [23] and Heo and Lee’s balanced benchmark of issue reports from eleven projects [14], each with 300 labeled and 300 test issues. Following Heo and Lee, both the retrieval index and the fine-tuning data come either from the target project’s 300 labeled issues or from all 3,300 labeled issues of the eleven projects. We compare the approaches in macro  $F_1$ , per-class errors, peak GPU memory, and time.

RAG shows the LLM the target project’s labeled issues most similar to the query (its neighbors) as examples. Two reference points separate what retrieval and the LLM each contribute. Zero-shot prompting is the LLM without examples. A similarity-weighted vote over the neighbors’ labels (the  $k$ NN voting baseline) is retrieval without the LLM and tests how much label information the neighbors carry. The vote alone reaches 59.5% macro  $F_1$ , so we give the neighbors to the LLM. As with prior IRC methods [13], [24], RAG’s most frequent error is labeling questions as bugs. We hypothesize that bug-labeled examples retrieved for a question reinforce this error, and a variant (filtered RAG) removes them when the neighbors suggest a question, that is, when bug-labeled neighbors do not clearly outnumber question-labeled ones.

We answer three research questions.

**RQ1:** *How well does retrieval-augmented few-shot prompting classify issue reports, and how many examples does it need?* With examples from the target project, RAG outperforms both zero-shot prompting and  $k$ NN voting at every number of examples, reaching 69.7–76.7% macro  $F_1$  at its best  $k$ . Its most frequent error is still labeling questions as bugs (26–36% of questions).

**RQ2:** *Does filtering the retrieved examples reduce question-to-bug errors?* Filtered RAG lowers the share of questions labeled as bugs to 19–27% and raises macro  $F_1$  over RAG by 1.5–2.4 points, at the cost of bug recall.

**RQ3:** *Can RAG match LoRA fine-tuning with less labeled data and GPU memory?* Given the same 300 labeled issues, both RAG variants outperform fine-tuning at every model size. Fine-tuning pulls ahead only with all 3,300 labeled issues of the eleven projects. It then leads filtered RAG by 1.0 point on average, a difference that is not statistically significant, and leads RAG by 2.8 points. Both RAG variants need no training and 33% less peak GPU memory on average, but their inference is slower. Fine-tuning finds more bugs but over-predicts the bug label.

Overall, with a simple filter on its examples, retrieval-augmented few-shot prompting preserves the macro  $F_1$  of LoRA fine-tuning with less labeled data per project, no training, and less GPU memory, and the two approaches make different errors. Our main contributions are the following:

- The first systematic evaluation of retrieval-augmented few-shot prompting for IRC, against LoRA fine-tuning of the same four models on the same labeled issues, covering macro  $F_1$ , per-class errors, labeled data, peak GPU memory, and time, with a replication package.
- Evidence that, with a simple filter on its examples, it matches fine-tuning’s macro  $F_1$  with  $11\times$  less labeled data per project, no training, and a third less GPU memory, and that it beats fine-tuning when both use the same labeled issues.
- A training-free example filter that reduces the question-to-bug confusion known from prior IRC work, with an analysis of the error trade-offs between the approaches.

Section II describes the approaches, Section III the setup, Section IV the results, Section V the failure analysis and implications, Section VI related work, and Section VII the threats to validity; Section VIII concludes.

## II. APPROACH

All retrieval-based classifiers share one retrieval step (Section II-B). The  $k$ NN voting baseline labels the query from its neighbors’ labels alone, RAG shows the neighbors to the LLM as labeled examples, and filtered RAG removes bug-labeled examples for suspected questions. We compare the two RAG variants with LoRA fine-tuning of the same LLMs (Section II-F); Fig. 1 shows all four approaches.

### A. Problem Formulation

Given a query issue report, we classify it into only one of the three labels: *bug*, *feature*, or *question*. Only the issue’s

title and description text are used for classification with no other signals from the issue tracker. The defined label space is adopted directly from prior IRC studies [6]–[8], [11], [19].

### B. Indexing and Retrieval

To prepare the issues for retrieval based on textual similarity, we first embed their textual content (i.e., *title* + *description*). We do not embed the issue labels. Instead, we retain them as metadata associated with each embedded issue. To generate the embeddings, we use the `all-MiniLM-L6-v2` model from the Sentence-Transformers library [25]. We selected this model due to its lightweight architecture, fast performance, and effectiveness on issue report text [26]. The embeddings are then indexed using the FAISS vector database library [27], which provides efficient high-dimensional similarity retrieval. At the retrieval stage, the query issue is first embedded with the same model. Next, FAISS calculates the cosine similarity of indexed issue reports to the query, and returns the top- $k$  nearest neighbors, ordered by descending similarity.

### C. $k$ NN Voting Baseline

The  $k$ NN voting baseline labels the query by a similarity-weighted vote over the labels of its top- $k$  neighbors, following the distance-weighted  $k$ -nearest-neighbor rule [28], [29].<sup>1</sup> Because it uses no LLM,  $k$ NN voting measures how much label information the neighbors carry. Its peak sets the largest  $k$  we give the LLM (Section IV-A), and it labels invalid LLM outputs in the fallback (Section IV-C). Formally, for a query issue  $q$  with retrieved neighbors  $\{(n_i, l_i, s_i)\}_{i=1}^k$ , where  $n_i$  is the  $i$ -th nearest neighbor with label  $l_i$  and cosine similarity  $s_i$  to  $q$ , the predicted label  $\hat{y}$  is given by:

$$\hat{y} = \arg \max_{c \in \mathcal{L}} \sum_{i=1}^k s_i \cdot \mathbf{1}[l_i = c], \quad (1)$$

where  $\mathcal{L}$  is the label set, and  $\mathbf{1}[l_i = c]$  is the indicator function that equals 1 when neighbor  $i$  has label  $c$  and 0 otherwise. For each label,  $k$ NN voting sums the cosine similarities of the neighbors that carry that label, and the  $\arg \max$  operator returns the label with the largest total sum. It breaks ties by returning the label of the most similar neighbor in the tied classes.

### D. Retrieval-Augmented Few-Shot Prompting (RAG)

RAG shows the query’s top- $k$  neighbors (Section II-B) to the LLM as labeled examples. We use the neighbors, rather than randomly chosen labeled issues, because similar examples can outperform random ones [22] and because the neighbors’ labels alone reach 59.5% macro  $F_1$  (Section IV-A). We create the few-shot prompt in three parts as shown in Fig. 2. First, the system message frames the classification task and the label space. Second, the top- $k$  nearest neighbors are listed in retrieval order (descending similarity), each formatted as *title* + *description* + *label*. Finally, the query issue closes the prompt,

<sup>1</sup>Squared-similarity weighting and uniform majority were also evaluated, but were outperformed by similarity-weighted voting.

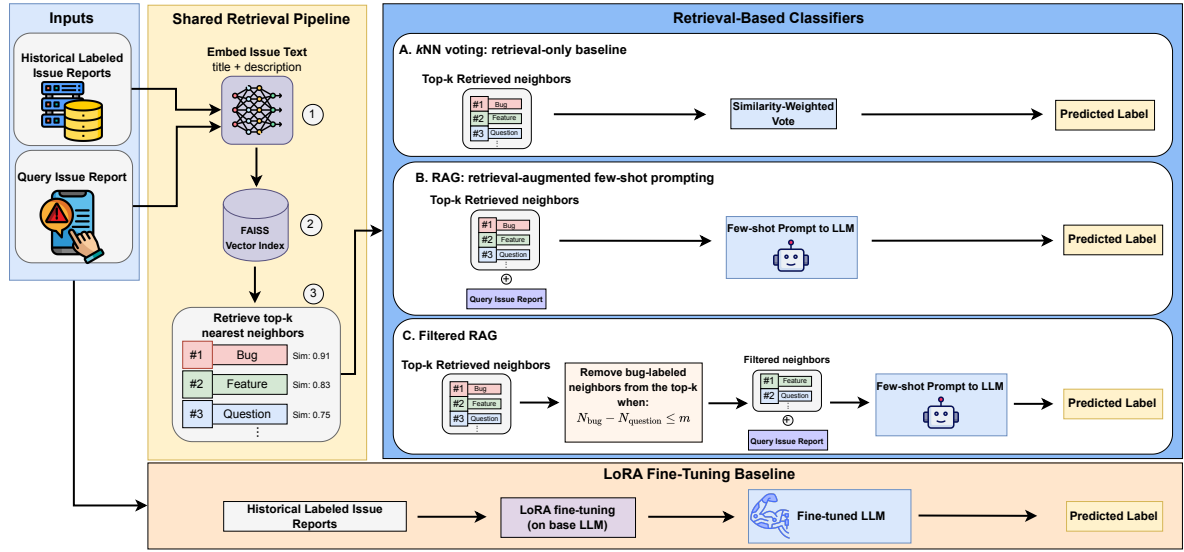


Fig. 1. Overview of the  $k$ NN voting baseline, retrieval-augmented few-shot prompting (RAG), filtered RAG, and the LoRA fine-tuning baseline.

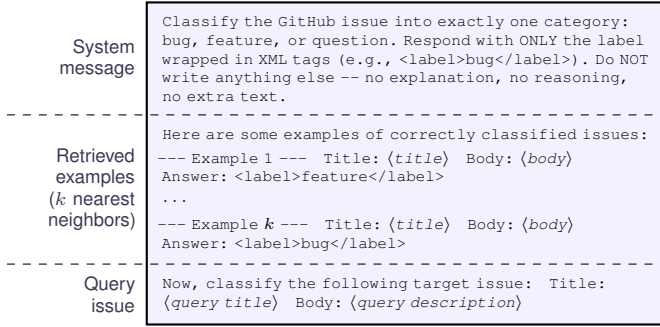


Fig. 2. The retrieval-augmented few-shot prompt structure used for RAG and filtered RAG.

since LLMs use information at the end of a long context better than in its middle [30]. With  $k=0$ , the prompt has no examples and RAG becomes zero-shot prompting, as in prior work [13].

We evaluate  $k \in \{0, 1, 3, 6, 9, 12, 15\}$ , up to  $k$ NN voting’s peaks at  $k=15-16$  (Section IV-A). The maximum prompt sequence length is set to 8,192 tokens.<sup>2</sup> Roughly 30% of the context quota is reserved for the query issue and 70% for the neighbors. If the neighbors exceed their overall quota, they are truncated proportionally based on their length (longer neighbors get more truncation). The query issue gets truncated if it exceeds its budget as well. Truncation happens on the right side of the description text, preserving all titles and labels (if any). The LLM is instructed to generate the predicted label wrapped in XML tags (e.g. `<label>bug</label>`). The labels of the presented examples also follow the same format. We use a regular expression to extract the predicted label. Outputs that contain none of the three labels are marked as

<sup>2</sup>Maximum prompt sequence length is selected empirically. 8,192 tokens offered the best trade-off between macro  $F_1$  and inference speed across our preliminary experiments.

*invalid*. Finally, models are loaded with Unsloth<sup>3</sup> for memory efficiency, and the temperature is set at 0.1 for deterministic classifications.

### E. Filtered RAG

RAG’s most frequent error is labeling questions as bugs (Section IV-A), a known weakness of IRC methods [13], [24]. The models make this error even at zero-shot, so we hypothesize that bug-labeled examples retrieved for a question reinforce it. Filtered RAG therefore removes the bug-labeled examples from the prompt when the neighbors suggest that the query is a question, that is, when  $N_{\text{bug}} - N_{\text{question}} \leq m$ , where  $N_{\text{label}}$  is the number of neighbors with that label among the top  $k$  and  $m$  is a small positive margin. For example, with  $k=12$  and  $m=3$ , a query whose neighbors are six bugs, four questions, and two features has  $N_{\text{bug}} - N_{\text{question}} = 2$ , so the six bug examples are removed and the prompt shows the remaining six.

We calibrated  $m$  before testing, on a validation split of the labeled issues. From each project’s 300 labeled issues, 30 issues stratified by label serve as queries and the other 270 as the retrieval index, mirroring the project-specific setting (Section III). On this split, true questions retrieve nearly balanced bug and question neighbors, with on average 1.36 more question than bug neighbors over  $k \in [1, 15]$ . We swept  $m \in \{1, \dots, 5\}$ , where 5 is the 95th percentile of  $N_{\text{bug}} - N_{\text{question}}$  for true questions, and chose  $m=3$  as the best trade-off between firing for questions and for bugs. Averaged over  $k$ , it fires for 89.1% of true questions and 56.7% of true bugs. Otherwise, filtered RAG is identical to RAG.

### F. LoRA Fine-Tuning Baseline

Prior LLM-based IRC studies fine-tune open LLMs on labeled issue reports with LoRA [6], [14], [20]. Each training example is an instruction prompt with the issue’s title and

<sup>3</sup><https://unsloth.ai>

description as input and its label as output. We adopt the prompt template of these studies [6], [14] verbatim, so that our baseline reproduces their method. At inference, the fine-tuned model receives the query issue in the same format and completes the empty label field. A regular expression extracts the label, and outputs that contain none of the three labels are *invalid*.

Training hyperparameters follow prior work where it reports them: LoRA rank and alpha of 16 and a 2,048-token limit [6], and 3 epochs with a per-device batch size of 1 [14]. We use a learning rate of  $2 \times 10^{-4}$ , the paged AdamW 8-bit optimizer [31], and 16 gradient-accumulation steps. The 2,048-token limit leaves 94.9% of the test issues untruncated; longer issues are truncated from the right of their description. Similar to RAG and filtered RAG, the models are loaded with Unsloth and the temperature is set at 0.1.

### III. EXPERIMENTAL SETUP

**Dataset and Settings:** We use the dataset introduced in Heo et al. [14] for our experiments. The dataset has a total of 6,600 issue reports gathered from eleven active open-source projects: `ansible`, `bitcoin`, `dart-lang`, `roslyn`, `react`, `flutter`, `kubernetes`, `TypeScript`, `vscode`, `opencv`, and `tensorflow`. Each project has 600 issue reports with an equal distribution across the labels (*bug*, *feature*, and *question*). To support their LLM fine-tuning method, Heo et al. divided the dataset into 50/50 train and test splits, stratified by project and label. This results in 300 train and 300 test issue reports per project. We also adopt Heo et al.’s project-specific (PS) and project-agnostic (PA) configurations to examine how data scope affects IRC performance. In the PS setting, each project’s train and test data are used in isolation, resulting in eleven different train-test pairs. In the PA setting, the train splits of all projects are merged into a single training set of 3,300 issue reports. In both configurations, RAG and  $k$ NN voting only use the training splits as their retrieval index to ensure fair comparison with the fine-tuning baseline.

**Models:** We use the Qwen2.5-Instruct model family [23] in our evaluations with four different parameter sizes: 3B, 7B, 14B, and 32B. Qwen2.5 is an open-source LLM with strong performance on general-purpose benchmarks. Using the same model with different parameter sizes enables analysis of the effect of model scale in LLM-based IRC approaches.

**Evaluation Metrics:** We report the macro-averaged  $F_1$  over the three labels as the headline metric, since it weighs the three labels equally. A method’s *best  $k$*  (its best configuration) is the one with the highest macro  $F_1$  on the test set. We report scores in percent and differences between methods in percentage points (points). We also report macro-averaged and per-class precision and recall. All metrics are calculated over the total 3,300-issue test set. We also report the rate of invalid outputs (*invalid rate*), which count as incorrect predictions. For significance, we report bootstrap 95% confidence intervals

(CIs) on the macro  $F_1$  differences between methods.<sup>4</sup>

**Hardware Setup:** Fine-tuning of Qwen-14B and Qwen-32B and the Qwen-32B runs at  $k \geq 12$  used one NVIDIA L40 or L40S GPU (48 GB VRAM). All other experiments used one NVIDIA RTX 4090 GPU (24 GB VRAM).

### IV. EVALUATION

Fig. 3 shows macro  $F_1$  across the  $k$  grid, and Table I lists every method at its best configuration with its per-class precision and recall. RQ1 evaluates RAG against its two reference points, RQ2 evaluates the example filter, and RQ3 compares both RAG variants with LoRA fine-tuning.

#### A. RQ1: Classification with Retrieved Examples

We compare RAG with two reference points. Zero-shot prompting ( $k=0$ ) is the LLM without examples and reaches 61.3–68.8% macro  $F_1$ . The  $k$ NN voting baseline is retrieval without the LLM. We evaluate it at  $k \in \{1, \dots, 20, 25, 30\}$  to find how many neighbors still add label information (Fig. 3a). Its macro  $F_1$  rises slowly with  $k$  and peaks at 59.5% with the target project’s issues ( $k=15$ ) and at 60.4% with all projects’ issues ( $k=16$ ); no larger  $k$  scores higher. These peaks set the top of RAG’s  $k$  grid.

The pooled index helps the vote only slightly. PA scores higher than PS at every  $k$ , but by little, because even the pooled index returns mostly neighbors from the query’s own project; PS and PA share 84–90% of their top- $k$  neighbors. The two settings therefore give the vote, and later the LLM, nearly the same neighbors.  $k$ NN voting also over-predicts bug and is weakest on questions, labeling 32% of them as bugs.

Without examples, the LLMs over-predict bug. Zero-shot prompting assigns the bug label to about half of all issues, although a third are bugs, and labels 46–59% of questions as bugs but far fewer as features. These questions make up 76–84% of its false bug predictions, so low question recall and low bug precision are two sides of the same error.

All 48 configurations with  $k \geq 1$  (four sizes, six values of  $k$ , two settings) outperform both reference points in macro  $F_1$ , macro precision, and macro recall (Fig. 3b). Even one example raises macro  $F_1$  at every size. The best  $k$  grows with model size, from 3 at Qwen-3B and 6 at Qwen-7B to 12 at Qwen-14B and Qwen-32B, giving 69.7–76.7% macro  $F_1$ . With examples, macro  $F_1$  grows with every step in model size, whereas zero-shot prompting does not (Qwen-14B scores below Qwen-7B). Beyond their best  $k$ , the two smaller models lose macro  $F_1$  as examples are added, while the two larger ones stay nearly flat; no model improves from  $k=12$  to  $k=15$ . We hypothesize that larger models make better use of long contexts with many examples. Retrieving from the target project scores slightly higher than retrieving from all projects at most sizes and values of  $k$ , as the neighbor overlap leads us to expect, so we use PS for RAG and filtered RAG in the rest of the paper.

<sup>4</sup>We resample the 3,300 test issues 1,000 times and evaluate both methods on the same resample (paired bootstrap).

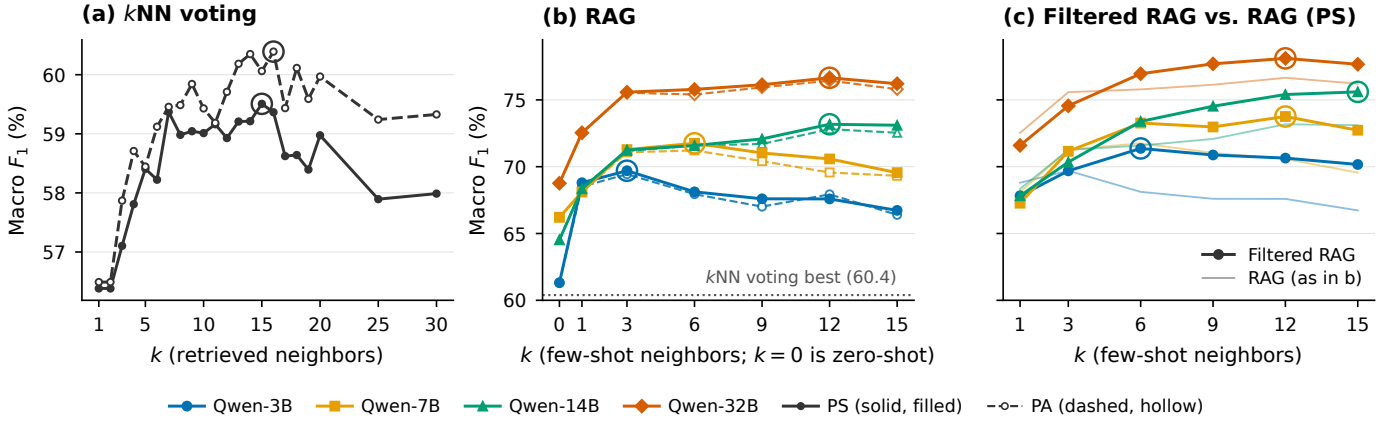


Fig. 3. Macro  $F_1$  (%) across the  $k$  grid. (a)  $k$ NN voting in PS (solid) and PA (dashed). (b) RAG for the four Qwen sizes in PS (solid, filled) and PA (dashed, hollow). (c) Filtered RAG (markers) against RAG. (b) and (c) share one  $y$ -axis and rings mark each method’s best  $k$ .

TABLE I  
POOLED TEST-SET RESULTS (%) OF EVERY METHOD AT ITS BEST CONFIGURATION.

| Model    | Method             | $k$ | Bug  |      | Feature |      | Question |      | $P_{\text{mac}}$ | $R_{\text{mac}}$ | Macro $F_1$ | Invalid |
|----------|--------------------|-----|------|------|---------|------|----------|------|------------------|------------------|-------------|---------|
|          |                    |     | $P$  | $R$  | $P$     | $R$  | $P$      | $R$  |                  |                  |             |         |
| –        | $k$ NN voting (PS) | 15  | 55.2 | 73.4 | 67.7    | 56.2 | 59.2     | 49.8 | 60.7             | 59.8             | 59.5        | 0.0     |
|          | $k$ NN voting (PA) | 16  | 55.6 | 73.1 | 68.9    | 57.9 | 60.0     | 50.8 | 61.5             | 60.6             | 60.4        | 0.0     |
| Qwen-3B  | Zero-shot          | 0   | 54.3 | 91.5 | 88.6    | 61.0 | 56.8     | 35.3 | 66.6             | 62.6             | 61.3        | 0.2     |
|          | RAG (PS)           | 3   | 63.8 | 80.3 | 79.4    | 81.3 | 69.4     | 49.4 | 70.9             | 70.3             | 69.7        | 0.2     |
|          | Filtered RAG (PS)  | 6   | 72.3 | 66.5 | 79.4    | 81.3 | 64.4     | 64.6 | 72.0             | 70.8             | <b>71.4</b> | 1.8     |
|          | Fine-tuning (PS)   | –   | 60.6 | 87.0 | 77.0    | 79.2 | 79.3     | 40.0 | 72.3             | 68.7             | 67.6        | 1.1     |
|          | Fine-tuning (PA)   | –   | 63.7 | 87.3 | 83.3    | 75.1 | 72.4     | 51.1 | 73.1             | 71.2             | 70.8        | 0.8     |
| Qwen-7B  | Zero-shot          | 0   | 57.5 | 88.1 | 79.2    | 78.9 | 78.3     | 36.7 | 71.7             | 67.9             | 66.2        | 0.1     |
|          | RAG (PS)           | 6   | 65.7 | 82.4 | 78.7    | 84.8 | 83.8     | 47.4 | 76.1             | 71.5             | 71.8        | 3.5     |
|          | Filtered RAG (PS)  | 12  | 71.3 | 77.4 | 76.1    | 85.5 | 82.5     | 55.8 | 76.6             | 72.9             | 73.8        | 3.8     |
|          | Fine-tuning (PS)   | –   | 65.3 | 63.5 | 75.9    | 76.9 | 61.9     | 62.7 | 67.7             | 67.7             | 67.7        | 0.1     |
|          | Fine-tuning (PA)   | –   | 67.2 | 87.8 | 86.8    | 82.5 | 80.0     | 58.8 | 78.0             | 76.4             | <b>76.2</b> | 0.3     |
| Qwen-14B | Zero-shot          | 0   | 57.0 | 87.6 | 76.1    | 85.5 | 87.3     | 29.4 | 73.5             | 67.5             | 64.5        | 0.1     |
|          | RAG (PS)           | 12  | 65.1 | 84.4 | 83.9    | 83.6 | 85.5     | 48.9 | 78.2             | 72.3             | 73.2        | 4.5     |
|          | Filtered RAG (PS)  | 15  | 69.6 | 80.3 | 83.6    | 84.4 | 83.2     | 57.9 | 78.8             | 74.2             | 75.6        | 4.7     |
|          | Fine-tuning (PS)   | –   | 70.0 | 74.0 | 82.8    | 69.2 | 60.9     | 66.9 | 71.2             | 70.0             | 70.4        | 0.3     |
|          | Fine-tuning (PA)   | –   | 73.0 | 86.5 | 88.2    | 78.1 | 76.4     | 71.0 | 79.2             | 78.5             | <b>78.5</b> | 0.0     |
| Qwen-32B | Zero-shot          | 0   | 61.1 | 86.1 | 76.6    | 86.9 | 85.7     | 38.7 | 74.5             | 70.6             | 68.8        | 0.2     |
|          | RAG (PS)           | 12  | 70.1 | 82.8 | 85.5    | 82.8 | 84.0     | 59.8 | 79.9             | 75.2             | 76.7        | 4.6     |
|          | Filtered RAG (PS)  | 12  | 74.8 | 80.4 | 84.7    | 83.3 | 80.8     | 66.4 | 80.1             | 76.7             | <b>78.1</b> | 4.0     |
|          | Fine-tuning (PS)   | –   | 74.4 | 70.0 | 81.4    | 82.5 | 66.5     | 69.4 | 74.1             | 74.0             | 74.0        | 0.1     |
|          | Fine-tuning (PA)   | –   | 68.9 | 92.9 | 83.2    | 87.6 | 89.5     | 53.4 | 80.5             | 78.0             | 77.1        | 0.1     |

PS: 300 labeled issues of the target project; PA: the 3,300 issues of all eleven projects.  $P/R$ : precision/recall;  $P_{\text{mac}}/R_{\text{mac}}$ : their macro averages. Invalid outputs count as incorrect. Bold: best macro  $F_1$  per model size.

Most of RAG’s gain comes from questions. At the best  $k$ , the examples add 5.5–8.7 points to zero-shot prompting, and the rise in question  $F_1$  accounts for 56–70% of this gain. The examples lower the share of questions labeled as bugs to 26–36%, and with it the share of issues labeled as bugs, which raises bug precision along with question recall. Question nevertheless keeps the lowest per-class  $F_1$  at every  $k$ , and questions labeled as bugs remain the largest source of false bug predictions (74–79%). RQ2 targets this error.

RAG outperforms both zero-shot prompting and  $k$ NN voting at every number of examples, reaching 69.7–76.7%

macro  $F_1$  at its best  $k$  (3 to 12). The gain comes mostly from questions, but 26–36% of questions are still labeled as bugs.

### B. RQ2: Filtering the Retrieved Examples

Fig. 3c compares filtered RAG with RAG at the same  $k$ . Filtered RAG scores higher at every  $k \geq 6$  for all four sizes and lower at  $k \leq 3$ . The margin explains the small values of  $k$ . With  $m=3$  and at most three neighbors,  $N_{\text{bug}} - N_{\text{question}} \leq 3$  holds for every query, so the filter removes the bug-labeled examples unconditionally. It leaves 40% of the prompts at  $k=1$  and 18%

at  $k=3$  without any example, which turns these queries into zero-shot prompts.

At each method’s best  $k$ , filtered RAG is 1.5–2.4 points higher than RAG, and the CIs of all four differences exclude zero. Its best  $k$  is larger at the three smaller sizes (6, 12, and 15 instead of 3, 6, and 12) and unchanged at Qwen-32B, and like RAG, it gains little or nothing beyond  $k=12$ . When the filter fires, it removes every bug-labeled example from the top  $k$ , so a larger initial  $k$  is needed to leave enough examples in the prompt.

The gain is a targeted reduction of question-to-bug errors. The share of questions labeled as bugs falls from 26–36% to 19–27% at the best  $k$ , and it also falls when averaged over all values of  $k$ , so the effect does not hinge on the choice of  $k$ . Question recall rises by 6.5–15.3 points, while question precision falls by 1.2–5.0 points. The precision loss comes mainly from bugs. The filter also fires for many true bugs (Section II-E), and the share of bugs labeled as questions rises at every size.

On the bug label, the filter trades recall for precision. Bug precision rises and bug recall falls at every size. The loss is largest at Qwen-3B, where bug recall drops from 80.3% to 66.5% and bug  $F_1$  drops by 1.8 points, the only size at which the filter over-corrects; at the three larger sizes, bug  $F_1$  rises. At Qwen-3B, the share of bugs labeled as questions nearly doubles, far more than at the larger sizes, which suggests that the smallest model follows the remaining examples most closely. Feature  $F_1$  changes by at most 1.1 points, so the filter leaves the feature label almost untouched (Table I gives all per-class values).

Filtered RAG outperforms RAG at every  $k \geq 6$ , by 1.5–2.4 points at each method’s best  $k$ . It lowers the share of questions labeled as bugs to 19–27%, at the cost of bug recall.

### C. RQ3: RAG versus LoRA Fine-Tuning

**Same labeled issues:** We fine-tune the four models in both settings (Table I). Given the same labeled issues of the target project, both RAG variants outperform fine-tuning at every size, RAG by 2.1–4.0 points and filtered RAG by 3.8–6.0 points. Fine-tuned on these issues, the models still improve on zero-shot prompting at every size, but less than with retrieved examples. We hypothesize that 300 issues are too few for fine-tuning to learn the task well, whereas RAG needs only the few that are most similar to each query. The large gain that fine-tuning draws from more training issues supports this.

Pooling the labeled issues of all eleven projects raises fine-tuning’s macro  $F_1$  by 3.1–8.4 points, consistent with Heo and Lee [14]. Pooling changes both the number of training issues and their project mix, and its effect differs by label; at Qwen-32B, for example, it raises bug recall but lowers question recall. Pooling does not help RAG, because the pooled index returns mostly the same neighbors (Section IV-A). We therefore compare each method in its better setting: the RAG variants with the target project’s issues (PS) and fine-tuning with the

TABLE II  
MACRO- $F_1$  DIFFERENCE FROM PA FINE-TUNING IN PERCENTAGE POINTS, WITH 95% CIs, WITHOUT AND WITH THE  $k$ NN VOTING FALLBACK.

| Model      | RAG          |              | Filtered RAG |              |
|------------|--------------|--------------|--------------|--------------|
|            | $\Delta F_1$ | 95% CI       | $\Delta F_1$ | 95% CI       |
| Qwen-3B    | −1.1         | [−2.8, +0.7] | +0.5         | [−1.2, +2.4] |
| + fallback | −1.5         | [−3.2, +0.2] | +0.9         | [−0.8, +2.8] |
| Qwen-7B    | −4.4         | [−5.9, −2.9] | −2.4         | [−3.9, −0.9] |
| + fallback | −3.3         | [−4.7, −1.7] | −1.1         | [−2.6, +0.4] |
| Qwen-14B   | −5.4         | [−6.8, −3.9] | −2.9         | [−4.4, −1.3] |
| + fallback | −4.0         | [−5.4, −2.5] | −1.4         | [−3.0, +0.1] |
| Qwen-32B   | −0.5         | [−2.0, +0.9] | +1.0         | [−0.5, +2.4] |
| + fallback | +0.8         | [−0.8, +2.2] | +2.1         | [+0.6, +3.5] |
| All sizes  | −2.8         | [−3.9, −1.8] | −1.0         | [−2.0, +0.1] |
| + fallback | −2.0         | [−3.0, −1.0] | +0.1         | [−0.9, +1.1] |

RAG and filtered RAG under PS at their best  $k$ , fine-tuning under PA; negative values favor fine-tuning. + fallback: invalid outputs of both methods labeled by  $k$ NN voting. Bold: CI excludes zero. All sizes: mean of the four per-model differences.

pooled issues (PA). The eleven PS indexes together hold the same 3,300 issues as the PA training set, so the settings differ in the labeled data available per target project, not in the total labeling effort.

**Pooled fine-tuning:** Averaged over the four sizes, filtered RAG trails fine-tuning by 1.0 point, and the CI includes zero; RAG trails by 2.8 points, and the CI excludes zero (Table II). The filter thus turns a significant deficit into a difference that is not significant. Per size, RAG trails fine-tuning everywhere, significantly at Qwen-7B and Qwen-14B. Fine-tuning is significantly ahead of filtered RAG at Qwen-7B and Qwen-14B, while filtered RAG is ahead at Qwen-3B and Qwen-32B, but not significantly. Fine-tuning’s significant leads fall at the two sizes where pooling helps it most, by more than 8 points, against about 3 points at Qwen-3B and Qwen-32B.

Each method’s best  $k$  is chosen on the test set, which favors the RAG variants, since fine-tuning has no  $k$  to select. To check how much this matters, we fix  $k=12$  for every size. Filtered RAG then trails fine-tuning by 1.2 points on average instead of 1.0, and it remains ahead at Qwen-32B.

Results also vary by project (Fig. 4). Filtered RAG matches or exceeds fine-tuning in 16 of the 44 (project, model) pairs, including all four sizes on `opencv` and three on `bitcoin`, while fine-tuning leads at all four sizes on `TypeScript`. Differences on single projects reach about 10 points in either direction, so the parity in aggregate does not hold for every project.

**Labeled data, memory, and time:** Filtered RAG uses  $11\times$  less labeled data per target project than pooled fine-tuning and has no training phase. For each target project, fine-tuning learns from 3,300 labeled issues, whereas the RAG variants retrieve from that project’s 300. Both RAG variants need 33% less peak GPU memory on average (Table III), comparing their inference with fine-tuning’s training and inference of the same model. The saving ranges from a quarter at Qwen-14B to almost half at Qwen-3B, although the RAG prompts may be four times as long as fine-tuning’s (Section II-F).

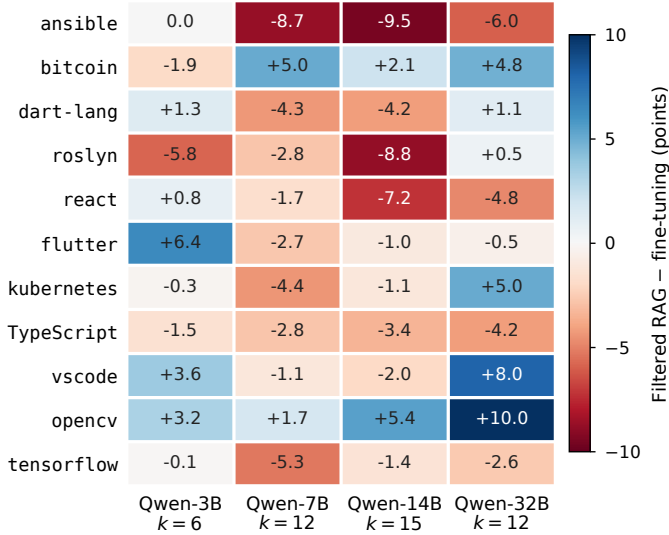


Fig. 4. Macro  $F_1$  difference in points between filtered RAG (PS, best  $k$ ) and PA LoRA fine-tuning on each project’s 300 test issues, for the four model sizes. Blue favors filtered RAG, red favors fine-tuning.

TABLE III

COMPUTATIONAL COST OF  $k$ NN VOTING (PS,  $k=15$ ), RAG, FILTERED RAG, AND LoRA FINE-TUNING AT THE CONFIGURATIONS OF TABLE II. RAM IS THE OBSERVED PEAK GPU MEMORY; TRAIN AND INFER ARE WALL-CLOCK RUNTIMES, AND TOTAL IS THEIR SUM.

| Model    | Method        | RAM (GB) | Train (h) | Infer (h) | Total (h) |
|----------|---------------|----------|-----------|-----------|-----------|
| –        | $k$ NN voting | 0.2      | –         | <0.01     | <0.01     |
| Qwen-3B  | RAG           | 2.9      | –         | 0.18      | 0.18      |
|          | Filtered RAG  | 2.9      | –         | 0.22      | 0.22      |
|          | Fine-tuning   | 5.5      | 0.31      | 0.11      | 0.42      |
| Qwen-7B  | RAG           | 6.8      | –         | 0.50      | 0.50      |
|          | Filtered RAG  | 6.8      | –         | 0.60      | 0.60      |
|          | Fine-tuning   | 9.9      | 0.48      | 0.10      | 0.58      |
| Qwen-14B | RAG           | 12.6     | –         | 1.37      | 1.37      |
|          | Filtered RAG  | 12.6     | –         | 1.35      | 1.35      |
|          | Fine-tuning   | 16.7     | 1.49      | 0.22      | 1.71      |
| Qwen-32B | RAG           | 22.3     | –         | 4.62      | 4.62      |
|          | Filtered RAG  | 22.3     | –         | 4.15      | 4.15      |
|          | Fine-tuning   | 31.5     | 2.28      | 0.44      | 2.71      |

The longer prompts make inference slower. Fine-tuned models label the 3,300 test issues in 0.10–0.44 h, and filtered RAG in 0.22–4.15 h. Including training, filtered RAG takes less total time than fine-tuning at Qwen-3B and Qwen-14B, about the same at Qwen-7B, and more at Qwen-32B. These totals include a single training run. Fine-tuning must be repeated whenever new labeled issues are to be learned or the model is replaced, whereas retrieval only indexes the new issues and uses any LLM as is. Conversely, fine-tuning’s faster inference pays off more, the more issues are labeled between two training runs. Without an LLM,  $k$ NN voting labels all test issues in under 4 s with 0.2 GB of GPU memory.

**Error profiles:** Fine-tuning over-predicts bug, as zero-shot prompting does (Section IV-A), although its training data are balanced across labels. It labels 39–46% of the test issues as

bugs, although 33% are bugs. It therefore finds more of the actual bugs than filtered RAG at every size, with bug recall of 86.5–92.9% against 66.5–80.4%, but it also makes more false bug predictions at three of the four sizes. The exception is Qwen-14B, where fine-tuning’s over-prediction is weakest. Per label, fine-tuning has the higher bug  $F_1$  at every size, whereas filtered RAG has the higher question  $F_1$  at Qwen-3B and Qwen-32B, the two sizes at which it leads in macro  $F_1$ .

The question-to-bug confusion persists under both approaches, with 23–36% of questions labeled as bugs after fine-tuning and 19–27% after filtered RAG. Our failure analysis (Section V-A) shows that most of the sampled Qwen-32B errors of the three LLM approaches are issues filed under a bug-report template. Which error profile fits a project better depends on whether a missed bug or a false bug label costs it more.

**Invalid outputs:** Invalid outputs make up 1.8–4.7% of filtered RAG’s outputs and at most 0.8% of fine-tuning’s. In a sample, most of them continue the issue body or the model’s reasoning instead of giving a label (Section V-A). An invalid output counts as a missed issue of its true label but adds no prediction to any label’s precision, so it lowers macro recall but not macro precision. Filtered RAG’s macro precision is within 1.4 points of fine-tuning’s at every size, while its macro recall is lower at Qwen-7B and Qwen-14B, where 6–8% of true bugs receive invalid outputs. We hypothesize that the long prompts, filled with issue text, lead the model to continue that text rather than answer.

To compare complete pipelines that give every issue a valid label, we let  $k$ NN voting label the invalid outputs of both methods. The RAG variants use PS voting at  $k=15$  over their unfiltered neighbors, and fine-tuning uses PA voting at  $k=16$ . The fallback needs no extra LLM call; the RAG variants reuse their neighbors, and fine-tuning gains a retrieval step. With the fallback, filtered RAG is 0.1 point ahead of fine-tuning on average, and the CI includes zero. Its deficits at Qwen-7B and Qwen-14B shrink to within 1.4 points and are no longer significant, and it becomes significantly ahead at Qwen-32B. Fine-tuning’s scores change by at most 0.3 points, since it has few invalid outputs, and RAG still trails it significantly on average.

With only the target project’s labeled issues, no training, and 33% less peak GPU memory on average, filtered RAG comes within 1.0 point of the macro  $F_1$  of fine-tuning on the labeled issues of all eleven projects, a difference that is not statistically significant; RAG trails by 2.8 points. Given the same labeled issues, both RAG variants outperform fine-tuning. Their inference is slower, and fine-tuning finds more bugs but over-predicts the bug label.

## V. DISCUSSION

### A. Failure Analysis

We manually analyzed the errors of Qwen-32B at each method’s best configuration. We randomly sampled 50 misclassified issues each from RAG, filtered RAG, and LoRA

fine-tuning, 150 in total. Each method’s sample has 30 true-question, 10 true-bug, and 10 true-feature errors, which mirrors the label distribution of the errors pooled over the three methods. Two authors analyzed each case independently and resolved disagreements through discussion. We found three categories.

**Wrong template (116 of 150, 77%):** The issue’s content and label contradict the template it was filed under. All of these errors are questions and feature requests filed under a bug-report template (*Steps to reproduce, Expected, Actual, System Information*), whose structure leads the LLM to predict bug regardless of the content. For example, `vscode#193985`<sup>5</sup> is a support question that opens with a literal `Type:Bug` because it uses VS Code’s bug template.

**Hybrid intent (20 of 150, 13%):** The issue combines intents and could defensibly carry more than one label, mostly because a fix or a feature request is layered into a question or bug report. For example, in `kubernetes#120364`<sup>6</sup> the author reports a concrete bug and, in the same body, requests a new feature: “*Please, give us multiarchitecture image! and fix monoarchitecture images too!*”

**Label noise (14 of 150, 9%):** The ground-truth label does not match the content. For example, `tensorflow#61710`<sup>7</sup>, titled “*Will there be an MLP model in the future version?*”, requests a new API feature but is labeled as a bug.

Wrong templates affect all three approaches. We hypothesize that the headings of a bug template are a strong cue for the bug label, both for the retriever, which then returns bug-labeled neighbors, and for the LLM itself. The other two categories are harder for any text-based classifier. For the 34 errors with hybrid intent or label noise (23%), another label is defensible or the ground truth is wrong, so they limit the macro  $F_1$  that any classifier can reach on this benchmark.

We also sampled 30 invalid outputs each from RAG and filtered RAG. In 44 of the 60 cases (73%), the model continued the issue body or its own reasoning. In 14 (23%), it produced degenerate text such as repeated digits and punctuation. In the remaining 2 (3%), it gave a label outside the three, such as `documentation`.

## B. Implications

**Labeled data and retraining:** With filtered RAG, a project’s own labeled issues give, without training and with 33% less peak GPU memory, a macro  $F_1$  within 1.0 point of fine-tuning on  $11\times$  as many issues pooled from eleven projects; the difference is not statistically significant, while RAG trails by 2.8 points (Section IV-C). The comparison is starker for a project that cannot draw on other projects’ labeled issues: fine-tuned on the same issues, the model scores 3.8–6.0 points below filtered RAG. Building the index for a project needs only its labeled issues and one embedding pass, whereas pooled fine-tuning needs labeled issues collected from other projects.

A new labeled issue then becomes an index update rather than a retraining run, so the examples can follow a project as its issues change, and the LLM can be replaced by a newer one without retraining.

**Where fine-tuning keeps an edge:** Fine-tuning keeps three advantages. It is significantly ahead at Qwen-7B and Qwen-14B, the sizes at which pooling the eleven projects’ issues helps it most (Section IV-C). Its inference is faster, most clearly at Qwen-32B, where filtered RAG takes longer in total even when fine-tuning’s training is included; this advantage grows with the number of issues labeled between two training runs. And it almost never produces invalid outputs, whereas the RAG variants need the  $k$ NN voting fallback to give every issue a valid label.

**Error profiles:** The approaches differ less in macro  $F_1$  than in which errors they make. Fine-tuning finds more bugs at every size, whereas filtered RAG raises fewer false bug alarms at three of the four sizes. Macro  $F_1$  weighs both errors equally, but their costs need not be equal: a false bug label costs a triager’s time, whereas a missed bug can delay a fix. Neither approach removes the question-to-bug confusion.

**Question-to-bug confusion:** The confusion persists under every method we test, including fine-tuning, as in prior IRC work (Section VI). The filter reduces it without training but cannot remove it. Many of the remaining errors are questions filed under a bug template (Section V-A), which read like bug reports to the retriever and the LLM alike.

**Retrieval alone as a floor:** Without an LLM,  $k$ NN voting reaches 59.5% macro  $F_1$ , 1.8 points below Qwen-3B’s zero-shot prompting, with 0.2 instead of 2.9 GB of GPU memory. Neither the neighbors’ labels nor the LLM alone reaches the scores of RAG, which outperforms both at every size (Section IV-A); the gain comes from giving the LLM the labeled neighbors.

**Classification and templates:** Our failure analysis shows that users often file support questions and feature requests under the bug-report template, which leads the classifier to predict bug regardless of the content. Most of the sampled errors are of this kind, so correcting the template when an issue is filed could prevent many of them. This matters for the LLM-assisted triage now offered on platforms such as GitHub [32] and GitLab [33], where the reporter describes the issue to an LLM that suggests a template and labels. Report validation matters as well, since mislabeled and duplicate reports are common [8], [15]. Prior studies validate bug reports [34]–[37], but combining classification and validation in one workflow remains underexplored.

## VI. RELATED WORK

**Automated Issue Report Classification (IRC):** Early IRC studies employed data mining and ML classifiers [7], [8], [10]. However, these methods yielded limited performance due to their shallow semantic understanding of issue report text [14]. To overcome this limitation, more recent work has adopted

<sup>5</sup><https://github.com/microsoft/vscode/issues/193985>

<sup>6</sup><https://github.com/kubernetes/kubernetes/issues/120364>

<sup>7</sup><https://github.com/tensorflow/tensorflow/issues/61710>

transformer models for their stronger contextual understanding [11], [16], [17], [38], [39]. These studies typically fine-tune the models on historical issues in a supervised setting. For instance, Izadi et al. [11] and Trautsch et al. [16] fine-tuned RoBERTa [40] and seBERT [41] models, respectively, to classify issues into bugs, features, or questions. Although transformer-based approaches achieve strong results, they rely on large amounts of training data, which limits their generalizability to settings where labeled data is scarce [11], [14], [17], [39]. These challenges have motivated the exploration of LLMs for IRC. Their extensive pretraining allows these models to be applied with little or no task-specific training, reducing the dependence on the large labeled datasets that supervised classifiers require [1], [6], [13], [19], [42]. However, state-of-the-art LLM-based methods do not leverage this property. Instead, they fine-tune the LLM with Low-Rank Adaptation (LoRA [20]) on labeled issue datasets to achieve their best reported performance [6], [14], [19]. Although fine-tuning is effective, it reintroduces the dependency on labeled training data, remains computationally expensive, and must be repeated whenever the underlying model is swapped or as newly labeled issues accumulate in the project [7], [10].

**Retrieval-Augmented Few-shot Prompting:** Studies show that few-shot prompting can improve LLM performance in specialized tasks due to in-context learning [21], [43]–[46]. Few-shot prompting has also been applied in IRC. Colavito et al. [1], [13] randomly selected one or two labeled examples per class from their training data and presented them in a few-shot prompt. In their benchmark, these examples did not improve over zero-shot prompting for most models [13]. Few-shot performance is sensitive to the quality of in-context examples. Research shows that using retrieval techniques to select the most semantically relevant in-context examples can outperform random selection [22], [47]. Since retrieval-augmented few-shot prompting can enhance performance without the overhead of training, it can be a promising alternative to the LoRA fine-tuning IRC baseline. Recent IRC studies also point in this direction in their future work. Heo and Lee [14] suggest few-shot prompting to improve accuracy across diverse projects, and Aracena et al. [6] propose retrieval augmentation, in which relevant past issues are retrieved and added to the LLM’s prompt, to reduce the need for extensive fine-tuning. To the best of our knowledge, however, this method has not been systematically and empirically evaluated for IRC.

**Question-to-Bug Misclassification:** In IRC studies that include both bug and question labels, question is often the hardest class to identify, and misclassified questions are most often labeled as bugs. For example, almost all methods in the NLBSE’24 tool competition struggled with question [12], and on the same dataset, the two best-performing zero-shot LLMs labeled 46–54% of questions as bugs [13]. Fine-tuned decoder LLMs also score lowest on question in aggregate [6], [14]. Our results show the same pattern. Every method we evaluate misclassifies questions as bugs, with  $k$ NN voting predicting bug for 32% of questions, zero-shot prompting for 46–59%,

and PA fine-tuning for 23–36% (Sections IV-A and IV-C). As a remedy, Aracena et al. [6] suggested retrieval augmentation, particularly for ambiguous labels such as question. Our RAG variants show that it helps but does not completely eliminate the problem. Without any training, retrieved examples lower the share of questions predicted as bugs from 46–59% to 26–36%, and filtered RAG lowers it further to 19–27%.

Prior work offers several explanations for why questions are misclassified as bugs. A common reason is label imbalance, since question is often the smallest class in the training data [11], [38]. Another explanation is that some questions read like bug reports because they describe unexpected behavior, contain error messages [24], or lack explicit interrogative markers [6]. Our failure analysis (Section V-A) shows the same effect for questions filed under a bug-report template, which led the model to predict bug regardless of their content. Finally, a label can depend on information outside the report, such as a maintainer’s later diagnosis [24]. Likewise, the label-noise cases in our failure analysis have labels that do not match their content and cannot be inferred from the report text alone.

## VII. THREATS TO VALIDITY

**Internal Validity:** Filtered RAG’s margin uses the same empirically derived fixed value across all models and projects with no per-project or per-model tuning, and is not framed as a universally optimal constant. Furthermore, the choice of hyperparameters and prompt format can also pose an internal threat to validity as different hyperparameters and prompts can change fine-tuning performance [6], [14].

**External Validity:** Issue report text can vary substantially in writing style, vocabulary, and structure across software projects [48], which may threaten the generalizability of our findings. Another threat to external validity is generalizability across settings. We note that many configurations, including LLM architecture, prompt structure, embedding model, and retrieval technique, can each lead to different results. Assessing how far our findings generalize to other settings is itself an important question, but one that falls outside this paper’s scope and warrants its own dedicated studies.

**Construct Validity:** The inherent randomness of LLM outputs can pose a potential threat to construct validity. The reported performance metrics can vary between different runs due to LLM randomness. To mitigate this variance, we ran the experiments three times and report the average.

## VIII. CONCLUSION AND FUTURE WORK

We studied retrieval-augmented few-shot prompting (RAG) as a training-free alternative to LoRA fine-tuning for issue report classification (IRC). We conducted an empirical study with four Qwen2.5-Instruct models of 3B to 32B parameters and 6,600 issue reports from eleven projects, taking the labeled issues either from the target project alone or from all eleven projects. Zero-shot prompting and a similarity-weighted vote over the neighbors’ labels ( $k$ NN voting) served as reference points for what the LLM and retrieval each contribute. Because

RAG, like prior methods, most often labels questions as bugs, we also evaluated a variant (filtered RAG) that removes bug-labeled examples when the neighbors suggest a question.

With this filter, retrieval-augmented few-shot prompting matches the macro  $F_1$  of LoRA fine-tuning while using  $11\times$  less labeled data per project, no training, and 33% less peak GPU memory on average. Using only the target project’s labeled issues, filtered RAG trails fine-tuning on the labeled issues of all eleven projects by 1.0 point, a difference that is not statistically significant, while RAG trails by 2.8 points. Per size, fine-tuning is significantly ahead at Qwen-7B and Qwen-14B, and filtered RAG is ahead at the other two sizes, but not significantly. Given the same labeled issues, both RAG variants outperform fine-tuning at every model size. These results build on two steps. Retrieved examples improve the LLM over both reference points at every number of examples, mostly on questions, and the filter further reduces question-to-bug errors at the cost of bug recall.

The two approaches make different errors. Fine-tuning finds more bugs but over-predicts the bug label, whereas filtered RAG makes fewer false bug predictions at most model sizes, and the question-to-bug confusion persists under every method we test. RAG’s inference is slower, and at Qwen-32B its total time exceeds fine-tuning’s even when training is included, while fine-tuning must be repeated whenever labels are added or the model is replaced. Future work can test these findings on other LLM families and quantization settings, on imbalanced, minority-class, and multi-label data, and on other issue trackers such as Jira and Bugzilla. Further directions are combining retrieval with a fine-tuned model, stronger retrieval-side interventions, such as LLM-based re-ranking of the neighbors, other embedding models and similarity metrics, and other prompt structures such as Chain-of-Thought prompting [49].

#### DATA AVAILABILITY

Our replication package [50] contains the code, data, and all necessary materials to reproduce the results presented in this paper.

## REFERENCES

- [1] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, "Leveraging gpt-like llms to automate issue labeling," in *Proceedings of the 21st International Conference on Mining Software Repositories*, 2024, pp. 469–480.
- [2] Z. Liao, D. He, Z. Chen, X. Fan, Y. Zhang, and S. Liu, "Exploring the characteristics of issue-related behaviors in github using visualization techniques," *IEEE Access*, vol. 6, pp. 24003–24015, 2018.
- [3] G. Catolino, F. Palomba, A. Zaidman, and F. Ferrucci, "Not all bugs are the same: Understanding, characterizing, and classifying bug types," *Journal of Systems and Software*, vol. 152, pp. 165–181, 2019.
- [4] H. Wu, Y. Ma, Z. Xiang, C. Yang, and K. He, "A spatial-temporal graph neural network framework for automated software bug triaging," *Knowledge-Based Systems*, vol. 241, p. 108308, 2022.
- [5] F. Santos, J. Vargovich, B. Trinkenreich, I. Santos, J. Penney, R. Britto, J. F. Pimentel, I. Wiese, I. Steinmacher, A. Sarma *et al.*, "Tag that issue: applying api-domain labels in issue tracking systems," *Empirical Software Engineering*, vol. 28, no. 5, p. 116, 2023.
- [6] G. Aracena, K. Luster, F. Santos, I. Steinmacher, and M. A. Gerosa, "Applying large language models to issue classification: Revisiting with extended data and new models," *Science of Computer Programming*, vol. 246, p. 103333, 2025.
- [7] R. Kallis, A. Di Sorbo, G. Canfora, and S. Panichella, "Ticket tagger: Machine learning driven issue classification," in *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2019, pp. 406–409.
- [8] G. Antoniol, K. Ayari, M. Di Penta, F. Khomh, and Y.-G. Guéhéneuc, "Is it a bug or an enhancement? a text-based approach to classify change requests," in *Proceedings of the 2008 conference of the center for advanced studies on collaborative research: meeting of minds*, 2008, pp. 304–318.
- [9] K. Herzig, S. Just, and A. Zeller, "It's not a bug, it's a feature: how misclassification impacts bug prediction," in *2013 35th international conference on software engineering (ICSE)*. IEEE, 2013, pp. 392–401.
- [10] Q. Fan, Y. Yu, G. Yin, T. Wang, and H. Wang, "Where is the road for issue reports classification based on text mining?" in *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 2017, pp. 121–130.
- [11] M. Izadi, "CatIss: An intelligent tool for categorizing issues reports using transformers," in *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, 2022, pp. 44–47.
- [12] R. Kallis, G. Colavito, A. Al-Kaswan, L. Pascarella, O. Chaparro, and P. Rani, "The NLBSE'24 tool competition," in *Proceedings of the Third ACM/IEEE International Workshop on NL-based Software Engineering*, 2024, pp. 33–40.
- [13] G. Colavito, F. Lanubile, and N. Novielli, "Benchmarking large language models for automated labeling: The case of issue report classification," *Information and Software Technology*, vol. 184, p. 107758, 2025.
- [14] J. Heo and S. Lee, "A study on applying large language models to issue classification," in *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*. IEEE Computer Society, 2025, pp. 1–11.
- [15] N. Pandey, D. K. Sanyal, A. Hudait, and A. Sen, "Automated classification of software issue reports using machine learning techniques: an empirical study," *Innovations in Systems and Software Engineering*, vol. 13, no. 4, pp. 279–297, 2017.
- [16] A. Trautsch and S. Herbold, "Predicting issue types with sebert," in *Proceedings of the 1st international workshop on natural language-based software engineering*, 2022, pp. 37–39.
- [17] G. Colavito, F. Lanubile, and N. Novielli, "Issue report classification using pre-trained language models," in *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, 2022, pp. 29–32.
- [18] J. Wang, X. Zhang, and L. Chen, "How well do pre-trained contextual language representations recommend labels for github issues?" *Knowledge-Based Systems*, vol. 232, p. 107476, 2021.
- [19] G. Aracena, K. Luster, F. Santos, I. Steinmacher, and M. A. Gerosa, "Applying large language models to issue classification," in *Proceedings of the Third ACM/IEEE International Workshop on NL-based Software Engineering*, 2024, pp. 57–60.
- [20] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, "Lora: Low-rank adaptation of large language models," *Iclr*, vol. 1, no. 2, p. 3, 2022.
- [21] T. Brown, B. Mann, N. Ryder *et al.*, "Language models are few-shot learners," *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [22] J. Liu, D. Shen, Y. Zhang, W. B. Dolan, L. Carin, and W. Chen, "What makes good in-context examples for gpt-3?" in *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd workshop on knowledge extraction and integration for deep learning architectures*, 2022, pp. 100–114.
- [23] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.
- [24] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, "Impact of data quality for automatic issue classification using pre-trained language models," *Journal of Systems and Software*, vol. 210, p. 111838, 2024.
- [25] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," *arXiv preprint arXiv:1908.10084*, 2019.
- [26] H. Huang, R. Widyasari, T. Zhang, I. C. Irsan, J. Shi, H. W. Ang, F. Liauw, E. L. Ouh, L. K. Shar, H. J. Kang *et al.*, "Back to the basics: Rethinking issue-commit linking with llm-assisted retrieval," *arXiv preprint arXiv:2507.09199*, 2025.
- [27] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The FAISS library," *IEEE Transactions on Big Data*, 2025.
- [28] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE transactions on information theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [29] S. A. Dudani, "The distance-weighted k-nearest-neighbor rule," *IEEE Transactions on Systems, Man, and Cybernetics*, no. 4, pp. 325–327, 1976.
- [30] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the association for computational linguistics*, vol. 12, pp. 157–173, 2024.
- [31] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *Advances in neural information processing systems*, vol. 36, pp. 10088–10115, 2023.
- [32] GitHub, "GitHub Copilot," <https://github.com/features/copilot>, 2025.
- [33] GitLab, "GitLab Duo," <https://about.gitlab.com/gitlab-duo/>, 2025.
- [34] L. Khaitib, N. S. Mathews, and M. Nagappan, "Assertflip: Reproducing bugs via inversion of llm-generated passing tests," *arXiv preprint arXiv:2507.17542*, 2025.
- [35] X. Wang, P. Gao, X. Meng, C. Peng, R. Hu, Y. Lin, and C. Gao, "Aegis: An agent-based framework for general bug reproduction from issue descriptions," *arXiv preprint arXiv:2411.18015*, 2024.
- [36] T. Ahmed, J. Ganhotra, R. Pan, A. Shinnar, S. Sinha, and M. Hirzel, "Otter: Generating tests from issues to validate swe patches," *arXiv preprint arXiv:2502.05368*, 2025.
- [37] E. Dinç and E. Tüzün, "Judge the votes: A system to classify bug reports and give suggestions," in *2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware)*. IEEE, 2025, pp. 01–10.
- [38] S. Bharadwaj and T. Kadam, "GitHub issue classification using BERT-style models," in *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, 2022, pp. 40–43.
- [39] M. Izadi, K. Akbari, and A. Heydarnoori, "Predicting the objective and priority of issue reports in software repositories," *Empirical Software Engineering*, vol. 27, no. 2, p. 50, 2022.
- [40] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.
- [41] J. Von der Mosel, A. Trautsch, and S. Herbold, "On the validity of pre-trained transformers for natural language processing in the software engineering domain," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1487–1507, 2022.
- [42] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, "Large language models for issue report classification," in *Ital-IA*, 2024, pp. 18–23.
- [43] M. Assi, S. Hassan, and Y. Zou, "Llm-cure: Llm-based competitor user review analysis for feature enhancement," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 4, pp. 1–25, 2026.
- [44] V.-H. Le and H. Zhang, "Log parsing with prompt-based few-shot learning," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2438–2449.
- [45] H. Ma, C. Zhang, Y. Bian, L. Liu, Z. Zhang, P. Zhao, S. Zhang, H. Fu, Q. Hu, and B. Wu, "Fairness-guided few-shot prompting for large language models," 2023. [Online]. Available: <https://arxiv.org/abs/2303.13217>

- [46] R. L. L. IV, I. Balažević, E. Wallace, F. Petroni, S. Singh, and S. Riedel, “Cutting down on prompts and parameters: Simple few-shot learning with language models,” 2021. [Online]. Available: <https://arxiv.org/abs/2106.13353>
- [47] G. Yu, L. Liu, H. Jiang, S. Shi, and X. Ao, “Retrieval-augmented few-shot text classification,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 6721–6735.
- [48] A. Akhavan, A. Hoseinpour, A. Heydarnoori, H. Bagheri, and M. Keshani, “LinkAnchor: An autonomous LLM-based agent for issue-to-commit link recovery,” in *To Appear in Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*, July 2026, preprint available at <https://arxiv.org/abs/2508.12232>.
- [49] A. Koyuncu, “Exploring fine-grained bug report categorization with large language models and prompt engineering: An empirical study,” *ACM Transactions on Software Engineering and Methodology*, 2025.
- [50] “Replication package: Can retrieval-augmented few-shot prompting match lora fine-tuning for issue report classification? an empirical study,” <https://figshare.com/s/2c7a6db860c611246774>, Accessed: 05-18-2026.